

4 - 指令系统及程序的机器级表示

作者：Klein White

时间：2025-04-21

概要：

冯·诺依曼结构机器对指令规定：

- 用二进制表示，和数据一起存放在主存
- 由两部分组成：操作码和操作数（或其地址码），操作码：定义操作的类型，操作数：指定源操作数或其地址

一条指令应该包含的信息：

- 操作码：指定操作的类型，其长度可以固定或可变
- 原操作数参照：指定一个或多个操作数的地址
- 结果值参照：计算结果的存放地址
- 下一条指令的地址：一般是隐含在 PC 里面，改变顺序时由指令给出 $PC=PC+1$

一条指令中的地址码类型：

零地址指令

- (1) 无需操作数 如：空操作 / 停机等
- (2) 所需操作数为默认的 如：堆栈 / 累加器等

形式：

OP

一地址指令

其地址既是操作数的地址，也是结果的地址

- (1) 单目运算：如：取反 / 取负等
- (2) 双目运算：另一操作数为默认的 如：累加器等

形式：

OP	A1
----	----

二地址指令（最常用）

分别存放双目运算中两个操作数，并将其中一个地址作为结果的地址。

形式：

OP	A1	A2
----	----	----

三地址指令（RISC风格）

分别作为双目运算中两个源操作数的地址和一个结果的地址。

形式：

OP	A1	A2	A3
----	----	----	----

多地址指令

用于成批数据处理的指令，如：向量 / 矩阵等运算的SIMD指令。

一条指令周期所做的事情：

- 从存储器取出指令
- 对指令译码，以确定将要做什么操作
- 计算操作数地址并取操作数
- 进行相应计算，并得到标志位
- 将计算结果保存到目的地
- 计算下条指令地址（通常和取指令同时进行） $PC=PC+1$

指令格式的选择应遵循的几条基本原则

- ◆ 应尽量短
- ◆ 要有足够的操作码位数
- ◆ 指令编码必须有唯一的解释，否则是不合法的指令
- ◆ 指令字长应是字节的整数倍
- ◆ 合理地选择地址字段的个数
- ◆ 指令尽量规整

一般通过对操作码进行不同的编码来定义不同的含义，操作码相同时，再由功能码定义不同的含义！

与指令集设计相关的重要方面

- ◆ 操作码的全部组成：操作码个数 / 种类 / 复杂度
LD/ST/INC/BRN 四种指令已足够编制任何可计算程序，但程序会很长
- ◆ 数据类型：对哪几种数据类型完成操作
- ◆ 指令格式：指令长度 / 地址码个数 / 各字段长度
- ◆ 通用寄存器：个数 / 功能 / 长度
- ◆ 寻址方式：操作数地址的指定方式
- ◆ 下条指令的地址如何确定：顺序，PC+1；条件转移；无条件转移；.....
- ◆ 异常和中断机制，包括存储保护方式等

指令中，操作数的基本类型有4个：地址、数值数据、字符/位串、布尔数据
数据可以放在寄存器或内存单元，也可以直接给出

寻址方式

指令或操作数地址的指定方式。即怎么根据地址找到指令或操作数。通常寻址方式特指“操作数的寻址”

对于指令的寻址

正常：PC = PC+1

跳跃方式：同操作数寻址，一般出现在一些判断指令中

操作数寻址方式

目的是形成操作数的有效地址。操作数可能存放的地方：指令直接给出、CPU寄存器、内存、IO端口。寻址方式可以分为：隐含寻址、立即寻址、直接寻址、间接寻址、寄存器寻址、寄存器间接寻址、变址寻址、基址寻址、相对寻址

隐含寻址

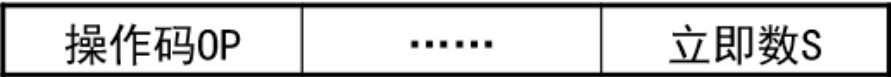
指令中隐含着操作数的地址。比如某些运算，直接隐含了以累加器 ACC 作为源、目的寄存器。也就是不指定操作数地址，默认使用某个数来计算

立即寻址

指令中直接把操作数的数值给出，操作数是指令的一部分，不能修改，灵活性较差。通常用于初始化

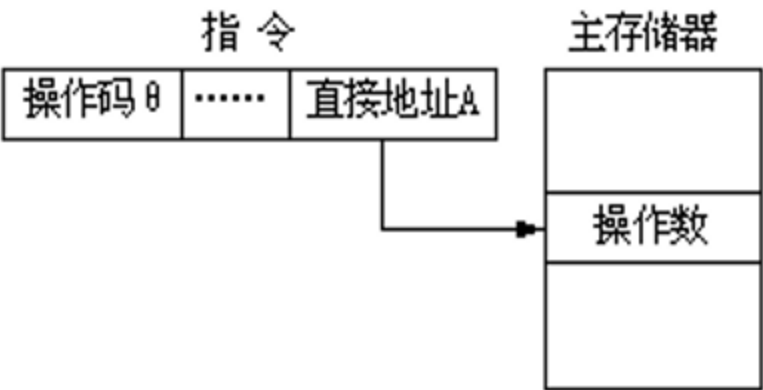
特点：在取指令时，操作码和操作数被同时取出，不必再次访问存储器，从而提高了指令的执行速度

指令格式：



直接寻址

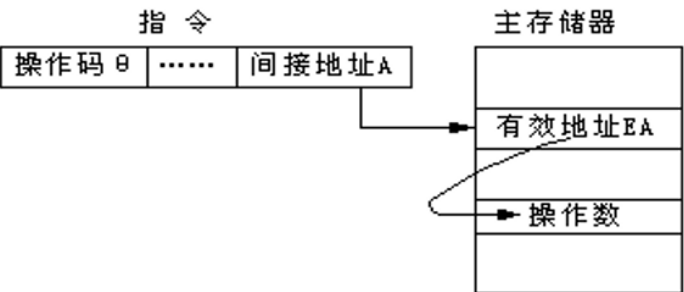
指令中地址码字段给出操作数的有效地址EA。操作数地址是不能修改的，又叫做绝对寻址方式



间接寻址

指令的地址码部分给出的地址A不是操作数的地址，而是操作数地址的地址

- 操作数的有效地址在主存储器中，可以被灵活的修改而不必修改指令，从而使间接寻址要比直接寻址灵活得多
 - 但是访问数据可能需要两次访存，速度较低
- $EA = (A), D = (EA) = ((A))$

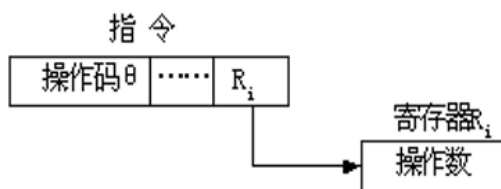


寄存器寻址

指令的地址码给出CPU内某一通用寄存器的编号，指令的操作数存放在相应的寄存器

- 寄存器在CPU的内部，指令在执行时从寄存器中取操作数比访问主存要快得多
- 寄存器数量少，这样寄存器编号占用的位数也会少，可以节约指令位数空间

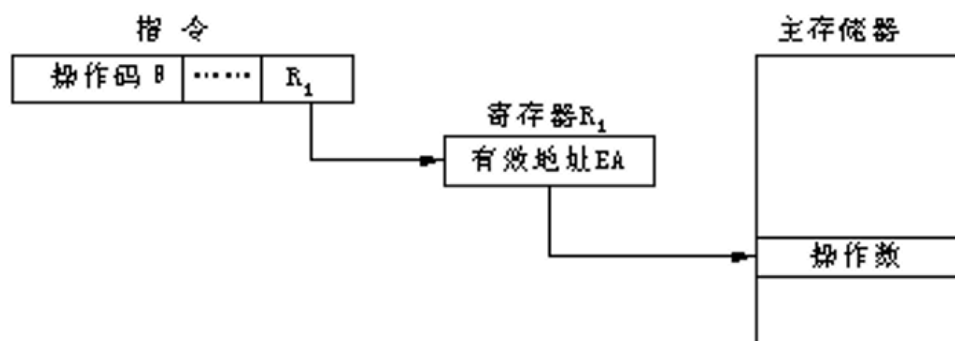
$EA=R_i, \quad D=(R_i)$



寄存器间接寻址

指令的地址码给出某通用寄存器编号，该寄存器内放操作数的地址，根据该地址到内存找到操作数

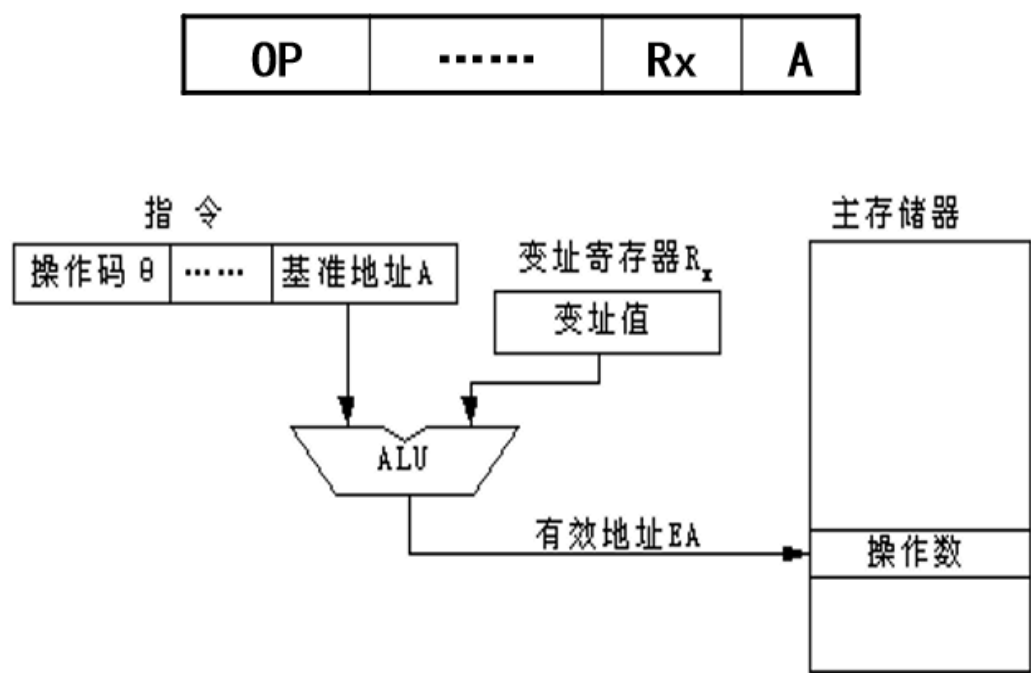
$EA=(R_i), \quad D=((R_i))$



变址寻址

指令的地址码部分给出的基准地址A与CPU内特定的变址寄存器R_x中的内容相加，以形成操作数的有效地址，即： $EA=A+(R_x)$

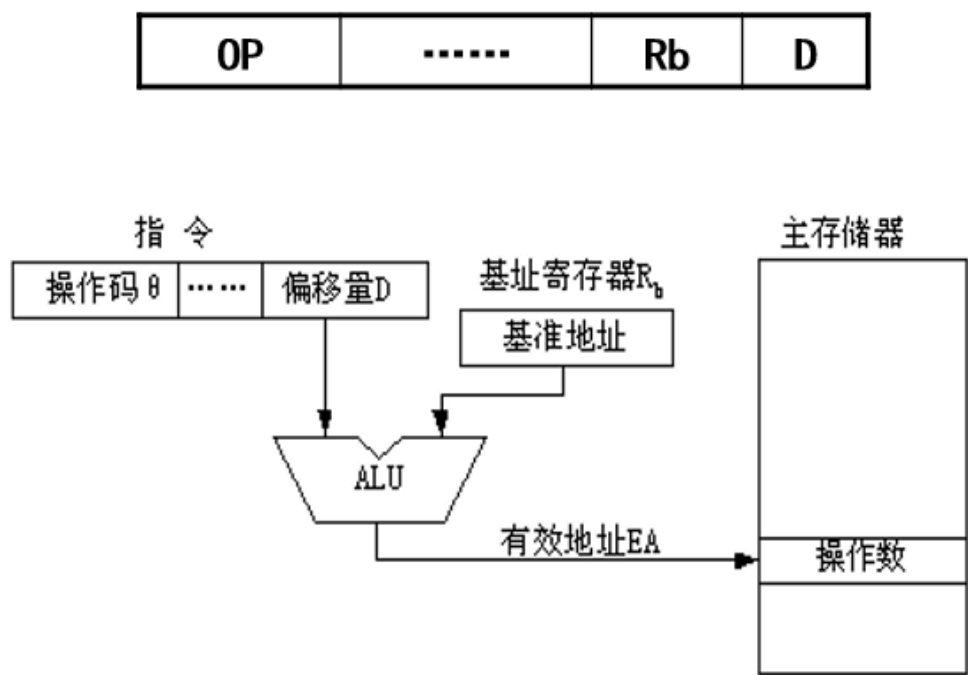
变址寄存器Rx中的内容称为变址值。



基址寻址

指令的地址码部分给出偏移量D，而基准地址放在基址寄存器Rb中，最后操作数的有效地址仍然是由基准地址与偏移量D相加而成，即： $EA = (Rb) + D$

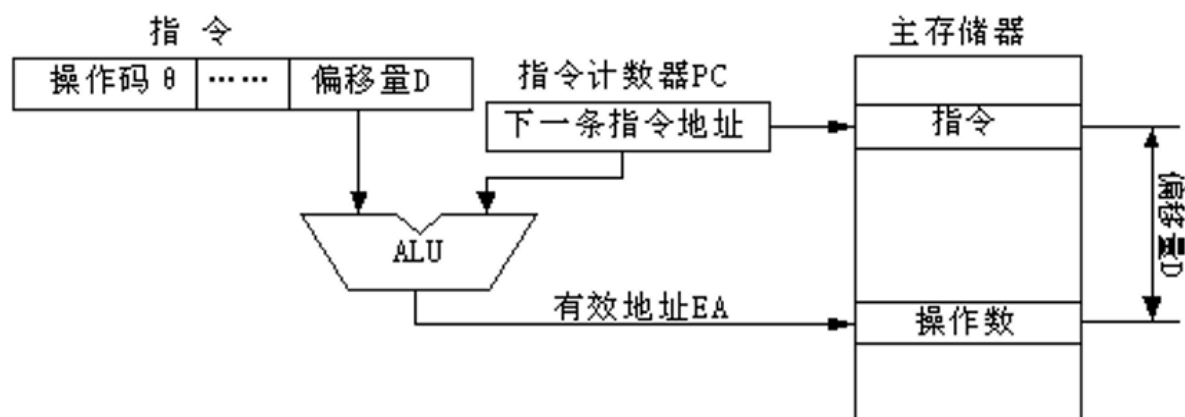
基址寄存器Rb中的内容称为基准地址。



相对寻址

指令的地址码给出相对的位移量D，程序计数器PC提供基准地址，两者相加后作为操作数的有效地址

◆ $EA = (PC) + D$ ， $S = ((PC) + D)$ ，D可正可负。



操作码编码有两种：定长操作码法、扩展操作码法

- **指令长度要求更高**时，变长的指令字和操作码会更好，但是可能存在**信息冗余**
- 性能更重要则选择**定长的指令字和操作码**，便于译码和访问

CISC 设计风格的特点：

- 指令系统复杂
- 指令周期长
- 各种指令都可以访问存储器
- 采用微程序控制
- 有专门的寄存器

RISC 设计风格特点：

- 简化的指令系统
- 以 RR 方式工作
- 指令周期更短
- 采用大量通用寄存器，访问内存少
- 采用组合逻辑电路控制，不用或少用微程序控制
- 采用优化的编译系统，力求有效地支持高级语言程序

RISC 指令的格式有6种：

共有6种指令格式

R-型为寄存器操作数指令

I-型为短立即数或装入（Load）指令

S-型为存储（Store）指令

B-型为条件跳转指令

U-型为长立即数操作指令

J-型为无条件跳转指令